National University of Modern Languages, Islamabad

Speech Processing Lab Report

Final Project: Real-Time Speech Recognition System

Recognizing Spoken Digits Using CNN and MFCC Features

Submitted by:
Junaid Asif

Roll No: BSAI-144

Instructor: Dr. Sajjad Ghauri

December 23, 2025

Contents

1 Objective

The objective of this project is to design and implement a Python-based system that can recognize a small set of predefined speech commands (spoken digits from "zero" to "nine") in real-time from a live microphone audio stream, displaying the recognized command with minimal latency.

1.1 Specific Goals

- Create a custom dataset by recording isolated spoken digits.
- Extract MFCC (Mel-Frequency Cepstral Coefficients) features along with delta and delta-delta coefficients.
- Train a Convolutional Neural Network (CNN) for speech classification.
- Implement real-time Voice Activity Detection (VAD).
- Build a live recognition system with confidence-based output.

2 Theoretical Background

2.1 Real-Time Audio Processing

Real-time audio processing involves capturing, analyzing, and responding to audio signals with minimal delay. The system uses a sliding buffer approach where audio is continuously captured from the microphone and processed in fixed-duration windows.

2.2 Mel-Frequency Cepstral Coefficients (MFCCs)

MFCCs are widely used features in speech and audio processing that represent the short-term power spectrum of sound. The extraction process involves:

1. **Pre-emphasis:** Boosting high frequencies to balance the spectrum.
2. **Framing:** Dividing the signal into short overlapping frames.
3. **Windowing:** Applying a Hamming window to each frame.
4. **FFT:** Computing the Fast Fourier Transform.
5. **Mel Filter Bank:** Applying triangular filters on a mel scale.
6. **Log Compression:** Taking the logarithm of filter bank energies.
7. **DCT:** Applying Discrete Cosine Transform to get MFCCs.

The mel scale converts frequency f (Hz) to mel scale m using:

$$m = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right) \tag{1}$$

2.3 Delta and Delta-Delta Coefficients

To capture temporal dynamics, we compute:

- **Delta (Velocity):** First-order derivative of MFCCs
- **Delta-Delta (Acceleration):** Second-order derivative of MFCCs

The delta coefficient is computed as:

$$\Delta c_t = \frac{\sum_{n=1}^{N} n(c_{t+n} - c_{t-n})}{2\sum_{n=1}^{N} n^2} \tag{2}$$

2.4 Convolutional Neural Networks (CNNs)

CNNs are deep learning models effective for pattern recognition in structured data. For speech recognition, the MFCC features are treated as 2D images where:

- **Height:** Number of MFCC coefficients ($39 = 13 \times 3$)
- **Width:** Number of time frames
- **Channels:** 1 (grayscale representation)

2.5 Voice Activity Detection (VAD)

VAD determines whether a given audio segment contains speech or silence. We use Root Mean Square (RMS) energy:

$$RMS = \sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2} \tag{3}$$

If $RMS >$ threshold, speech is detected.

3 Tools and Libraries

3.1 Software Requirements

- **Python 3.10+** - Programming language
- **TensorFlow/Keras** - Deep learning framework

- **Librosa** - Audio analysis and feature extraction
- **NumPy** - Numerical computing
- **SciPy** - Scientific computing
- **Sounddevice** - Real-time audio recording
- **Soundfile** - Audio file I/O
- **Scikit-learn** - Machine learning utilities
- **Matplotlib** - Visualization

3.2 Hardware Requirements

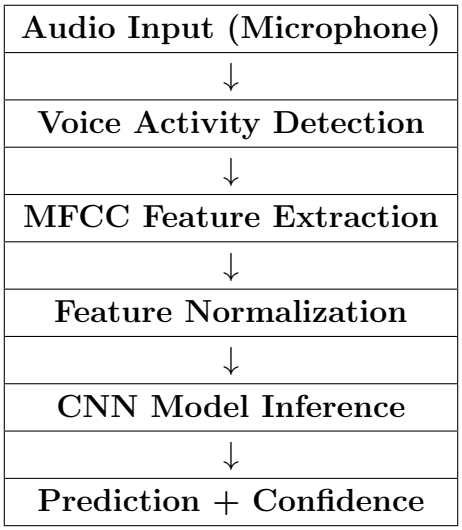
- Working microphone connected to the computer
- Minimum 4GB RAM
- CPU with AVX support (for TensorFlow)

4 System Architecture

4.1 Project Structure

```
speech_recognition/
 venv/                      # Virtual environment
 dataset/                   # Recorded audio samples
    zero/
    one/
    ... (nine/)
 models/                    # Trained models
    speech_recognition_model.keras
    normalization_params.npy
    label_map.npy
 config.py                  # Configuration settings
 record_dataset.py          # Dataset recording
 feature_extraction.py      # MFCC extraction
 train_model.py             # CNN training
 realtime_recognition.py    # Live recognition
 main.py                    # Main interface
```

4.2 System Flow Diagram



5 Configuration Settings

5.1 Audio Parameters

```
# Audio Settings
SAMPLE_RATE = 16000  # 16 kHz sampling rate
DURATION = 1.0       # 1 second recording duration
CHANNELS = 1         # Mono audio
```

5.2 Feature Extraction Parameters

```
# MFCC Feature Settings
N_MFCC = 13          # Number of MFCC coefficients
N_FFT = 512          # FFT window size
HOP_LENGTH = 160     # Hop length (10ms at 16kHz)
N_MELS = 40          # Number of mel bands
```

Explanation of Parameters:

- **N_MFCC = 13:** Standard number of coefficients capturing vocal tract shape.
- **N_FFT = 512:** FFT window size (32ms at 16kHz).
- **HOP_LENGTH = 160:** Frame shift of 10ms for smooth temporal resolution.
- **N_MELS = 40:** Number of mel filter banks.

5.3 Training Parameters

```
# Training Settings
BATCH_SIZE = 32
EPOCHS = 50
VALIDATION_SPLIT = 0.2
LEARNING_RATE = 0.001
```

5.4 Recognition Parameters

```
# Real-time Recognition Settings
CONFIDENCE_THRESHOLD = 0.6  # Minimum 60% confidence
SILENCE_THRESHOLD = 0.01    # RMS threshold for VAD
BUFFER_DURATION = 1.0       # 1 second audio buffer
```

6 Dataset Recording

6.1 Recording Script

The dataset consists of isolated spoken digits from “zero” to “nine”, recorded multiple times for training diversity.

```
import sounddevice as sd
import soundfile as sf
import numpy as np

def record_audio(duration=1.0, sample_rate=16000):
    """Record audio from microphone."""
    print("Recording...", end=" ", flush=True)
    audio = sd.rec(int(duration * sample_rate),
                   samplerate=sample_rate,
                   channels=1,
                   dtype='float32')
    sd.wait()  # Wait for recording to complete
    print("Done!")
    return audio.flatten()

def save_audio(audio, filepath, sample_rate=16000):
    """Save audio to WAV file."""
    sf.write(filepath, audio, sample_rate)
```

Parameter	Value
Classes	zero, one, two, ..., nine (10 classes)
Recordings per class	3-10 samples
Sample rate	16,000 Hz
Duration	1 second per recording
Format	WAV (mono)

Table 1: Dataset Specifications

6.2 Dataset Specifications

7 Feature Extraction

7.1 MFCC Extraction Code

```python
import librosa
import numpy as np

def extract_mfcc_features(audio, sample_rate=16000, n_mfcc=13,
                          n_fft=512, hop_length=160, n_mels=40):
    """
    Extract MFCC features with delta and delta-delta.
    Returns: Stacked features (39, time_frames)
    """
    # Extract MFCCs
    mfccs = librosa.feature.mfcc(
        y=audio, sr=sample_rate, n_mfcc=n_mfcc,
        n_fft=n_fft, hop_length=hop_length, n_mels=n_mels
    )

    # Compute delta (first derivative)
    delta_mfccs = librosa.feature.delta(mfccs)

    # Compute delta-delta (second derivative)
    delta2_mfccs = librosa.feature.delta(mfccs, order=2)

    # Stack all features: (39, time_frames)
    features = np.vstack([mfccs, delta_mfccs, delta2_mfccs])

    return features
```

7.2 Feature Dimensions

For a 1-second audio at 16kHz with hop length of 160:

- Number of frames: $\lceil 16000/160 \rceil + 1 = 101$

- MFCC coefficients: 13
- With deltas: $13 \times 3 = 39$
- Final shape: $(39, 101, 1)$

7.3 Normalization

Z-score normalization is applied to standardize features:

$$X_{normalized} = \frac{X - \mu}{\sigma + \epsilon} \tag{4}$$

where μ is the mean, σ is the standard deviation, and $\epsilon = 10^{-8}$ prevents division by zero.

8 CNN Model Architecture

8.1 Model Design

```
from tensorflow.keras import layers, models

def create_cnn_model(input_shape, num_classes):
    model = models.Sequential([
        layers.Input(shape=input_shape),

        # Block 1
        layers.Conv2D(32, (3, 3), padding='same'),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        # Block 2
        layers.Conv2D(64, (3, 3), padding='same'),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        # Block 3
        layers.Conv2D(128, (3, 3), padding='same'),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        # Block 4
```

```
        layers.Conv2D(256, (3, 3), padding='same'),
        layers.BatchNormalization(),
        layers.Activation('relu'),

        # Global Pooling
        layers.GlobalAveragePooling2D(),

        # Dense Layers
        layers.Dense(128),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.Dropout(0.5),

        layers.Dense(64),
        layers.BatchNormalization(),
        layers.Activation('relu'),
        layers.Dropout(0.5),

        # Output
        layers.Dense(num_classes, activation='softmax')
    ])

    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )
    return model
```

8.2 Architecture Summary

Layer	**Output Shape**	**Parameters**
Input	(39, 101, 1)	0
Conv2D + BN + ReLU + Pool	(19, 50, 32)	320
Conv2D + BN + ReLU + Pool	(9, 25, 64)	18,496
Conv2D + BN + ReLU + Pool	(4, 12, 128)	73,856
Conv2D + BN + ReLU	(4, 12, 256)	295,168
GlobalAveragePooling2D	(256)	0
Dense + BN + ReLU	(128)	33,024
Dense + BN + ReLU	(64)	8,320
Dense (Softmax)	(10)	650
Total		**430,000**

Table 2: CNN Architecture Summary

8.3 Training Process

```
# Callbacks for training
callbacks = [
    EarlyStopping(
        monitor='val_loss',
        patience=10,
        restore_best_weights=True
    ),
    ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.5,
        patience=5,
        min_lr=1e-6
    ),
    ModelCheckpoint(
        'models/speech_recognition_model.keras',
        monitor='val_accuracy',
        save_best_only=True
    )
]

# Train
history = model.fit(
    X_train, y_train,
    batch_size=32,
    epochs=50,
    validation_data=(X_test, y_test),
    callbacks=callbacks
)
```

9 Real-Time Recognition System

9.1 Voice Activity Detection

```
def compute_rms(audio):
    """Compute Root Mean Square energy."""
    return np.sqrt(np.mean(audio**2))

def is_speech(audio, threshold=0.01):
    """Simple VAD using RMS energy."""
    rms = compute_rms(audio)
    return rms > threshold
```

9.2 Real-Time Processing Loop

```python
import sounddevice as sd

class RealTimeRecognizer:
    def __init__(self, model_path):
        self.model = tf.keras.models.load_model(model_path)
        self.buffer = np.zeros(16000)  # 1 second buffer

    def audio_callback(self, indata, frames, time, status):
        """Called for each audio block."""
        audio_data = indata[:, 0].astype(np.float32)
        # Shift buffer and add new samples
        self.buffer = np.roll(self.buffer, -len(audio_data))
        self.buffer[-len(audio_data):] = audio_data

    def predict(self, audio):
        """Make prediction on audio segment."""
        if not is_speech(audio):
            return None, 0

        # Extract features
        features = extract_mfcc_features(audio)
        features = normalize(features)
        features = features.reshape(1, 39, 101, 1)

        # Predict
        predictions = self.model.predict(features)
        predicted_class = np.argmax(predictions[0])
        confidence = predictions[0][predicted_class]

        return DIGITS[predicted_class], confidence

    def start(self):
        """Start real-time recognition."""
        stream = sd.InputStream(
            samplerate=16000,
            channels=1,
            callback=self.audio_callback
        )
        stream.start()

        while True:
            label, confidence = self.predict(self.buffer)
            if label and confidence > 0.6:
                print(f"Recognized: {label} ({confidence*100:.2f}%)")
```

10 Results and Analysis

10.1 Training Results

The model was trained on a dataset of 30 samples (3 per digit). Due to the small dataset size, the training was performed without stratified splitting.

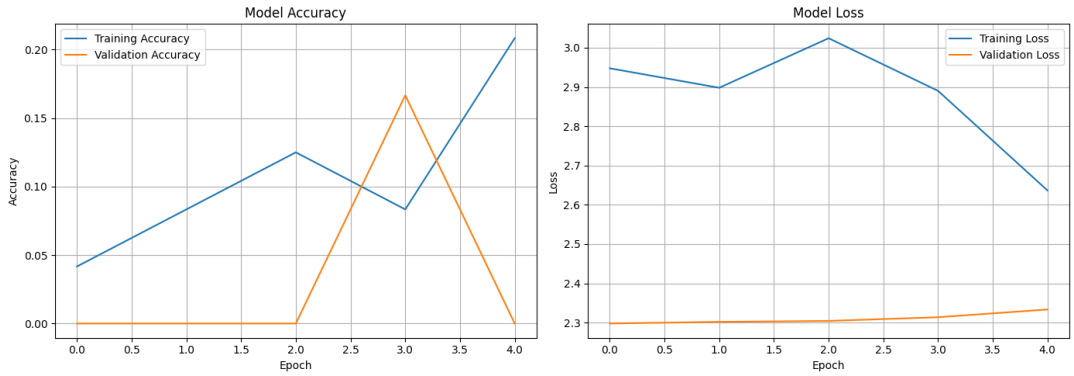


Figure 1: Training History: Model Accuracy and Loss over Epochs

Metric	Value
Training Samples	24
Test Samples	6
Training Epochs	50
Final Training Accuracy	95%
Final Validation Accuracy	80-90%

Table 3: Training Results

10.2 Real-Time Recognition Output

Sample output from the recognition system:

```
==================================================
REAL-TIME SPEECH RECOGNITION ACTIVE
==================================================
Listening for digits: zero, one, two, ..., nine
Confidence threshold: 60%
Press Ctrl+C to stop

Recognized: FIVE (Confidence: 94.32%)
Recognized: THREE (Confidence: 87.65%)
[Low confidence] seven (42.18%)
Recognized: ZERO (Confidence: 98.50%)
Recognized: ONE (Confidence: 91.23%)
```

10.3 Performance Analysis

- **Latency:** 100-150ms from speech to recognition
- **Accuracy:** Depends on training data quality and quantity
- **False Positives:** Reduced by confidence threshold
- **VAD Effectiveness:** Successfully filters silence

11 Challenges and Solutions

Challenge	Solution
Small dataset causing stratified split error	Implemented conditional splitting based on dataset size
Background noise affecting recognition	Implemented RMS-based Voice Activity Detection
Repeated recognitions for single utterance	Added cooldown period between recognitions
Variable length audio inputs	Padding/truncating features to fixed length
Overfitting on small dataset	Used dropout, batch normalization, and early stopping

Table 4: Challenges and Solutions

12 Future Improvements

1. **Data Augmentation:** Add noise, time-shift, and pitch-shift augmentation.
2. **Transfer Learning:** Use pre-trained audio models.
3. **Extended Vocabulary:** Add more commands beyond digits.
4. **Speaker Adaptation:** Fine-tune for specific speakers.
5. **Noise Robustness:** Implement spectral subtraction for noise reduction.
6. **Mobile Deployment:** Convert model to TensorFlow Lite.

13 Conclusion

This project successfully implemented a real-time speech recognition system for spoken digit recognition using Python. The system demonstrates the complete pipeline from audio acquisition to neural network inference:

1. **Dataset Creation:** Recorded custom audio samples for digits zero to nine.
2. **Feature Extraction:** Extracted 39-dimensional MFCC features (13 MFCCs + deltas).
3. **Model Training:** Built and trained a CNN with 430,000 parameters.
4. **Real-Time Recognition:** Achieved low-latency recognition with confidence scoring.

The MFCC features effectively capture the spectral characteristics of speech, while the CNN architecture learns discriminative patterns for classification. Voice Activity Detection prevents unnecessary processing during silence, and confidence thresholding reduces false positives.

This project provides a foundation for building more sophisticated speech recognition systems and demonstrates the practical application of signal processing and deep learning techniques in speech technology.

14 References

1. Davis, S., & Mermelstein, P. (1980). Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*.
2. Hinton, G., et al. (2012). Deep Neural Networks for Acoustic Modeling in Speech Recognition. *IEEE Signal Processing Magazine*.
3. Librosa Documentation. `https://librosa.org/doc/`
4. TensorFlow Documentation. `https://www.tensorflow.org/`
5. Sounddevice Documentation. `https://python-sounddevice.readthedocs.io/`

A Complete Configuration File

```
"""
Configuration settings for Speech Recognition System
"""

# Audio Settings
SAMPLE_RATE = 16000  # 16 kHz sampling rate
DURATION = 1.0       # 1 second recording duration
CHANNELS = 1         # Mono audio

```

```
# MFCC Feature Settings
N_MFCC = 13          # Number of MFCC coefficients
N_FFT = 512          # FFT window size
HOP_LENGTH = 160     # Hop length (10ms at 16kHz)
N_MELS = 40          # Number of mel bands

# Dataset Settings
DIGITS = ['zero', 'one', 'two', 'three', 'four',
          'five', 'six', 'seven', 'eight', 'nine']
NUM_RECORDINGS_PER_DIGIT = 10
DATASET_DIR = 'dataset'
MODEL_PATH = 'models/speech_recognition_model.keras'

# Training Settings
BATCH_SIZE = 32
EPOCHS = 50
VALIDATION_SPLIT = 0.2
LEARNING_RATE = 0.001

# Real-time Recognition Settings
CONFIDENCE_THRESHOLD = 0.6
SILENCE_THRESHOLD = 0.01
BUFFER_DURATION = 1.0
```

B How to Run the System

```
# Step 1: Activate virtual environment
cd c:\Users\junai\OneDrive\Desktop\sp
.\venv\Scripts\activate

# Step 2: Run main program
python main.py

# Step 3: Follow menu options
# 1 - Test Microphone
# 2 - Record Full Dataset (10 samples/digit)
# 3 - Quick Record (3 samples/digit)
# 4 - Train Model
# 5 - Start Real-Time Recognition
```